

1. **Rules and selectors:** Writing and linking CSS · Basic selectors: type, class, id, attribute · Combinators: descendant, child and siblings · Pseudo-classes: state and position
2. **Cascade and specificity:** The cascade: which declaration wins · Specificity: the (a, b, c) score · Inheritance: values that flow down the tree
3. **Box model and flow:** The box model: content, padding, border, margin · box-sizing: what width really measures · Margins: centring with auto, and collapsing · Normal flow and display: block, inline, inline-block, none
4. **Units and styling:** Units: px, %, em, rem, vw/vh and fr · Colour, text and decoration
5. **Flexbox:** Flexbox: container, items and the two axes · Flexbox alignment: justify-content, align-items, gap, wrap · Flexible sizes: flex-grow, flex-shrink, flex-basis
6. **Grid:** Grid: tracks, fr and auto-placement · Placing grid items: lines, spans and alignment · Named areas: grid-template-areas
7. **Responsive design:** Responsive foundations: viewport tag, fluid widths, fluid images · Media queries and breakpoints · Flexbox or Grid?

Rules and selectors

Writing and linking CSS

A CSS rule pairs a selector (which elements) with a declaration block of `property: value;` **pairs (how they look).**

Rules reach the page inline, in a `<style>` element, or from an external `.css` file linked with `<link rel="stylesheet">`.

- **Inline:** a `style="color: red"` attribute on one element. Affects only that element and beats every selector, so it is hard to override later. Avoid it outside quick tests.
- **Internal:** a `<style>` element inside `<head>`. Styles one page only.
- **External:** `<link rel="stylesheet" href="styles.css">` inside `<head>`. One file shared by every page of the site: the standard way.

```

<!-- index.html, fixtures.html, login.html: the same line in every <head> -->
<head>
  <meta charset="UTF-8">
  <title>Riverside FC</title>
  <link rel="stylesheet" href="styles.css">
</head>

/* styles.css: one file, every page */
nav a {
  color: #1A1F6C;          /* text colour */
  text-decoration: none; /* no underline */
}

```

Example. A club site has `index.html`, `fixtures.html` and `login.html`, each with the same `<link rel="stylesheet" href="styles.css">`. Changing `nav a { color: ... }` once in `styles.css` recolours the navigation on all three pages, and the browser downloads the file once and reuses it from its cache on the next page.

Common mistake. Writing `<style src="styles.css">` or putting the `<link>` in `<body>`. `<style>` has no `src` attribute: external CSS only loads through `<link rel="stylesheet">` in the `<head>`. Check the path too: `href="css/styles.css"` is relative to the HTML file, and a wrong path fails silently (the Network tab in DevTools shows a 404).

Basic selectors: type, class, id, attribute

A selector picks the elements a rule applies to: by tag name (p), by class (.card), by id (#hero), by attribute (a[target]) or all of them (*). Selectors can be compounded (article.card) and grouped with commas (h1, h2).

The basic selectors

Selector	Matches	Example
Type	every element with that tag	p, nav, button
Class	every element whose class list contains the name	.card, .btn-primary
Id	the one element with that id	#hero
Attribute	elements that have an attribute, or a given value	a[target], input[type="email"]

Selector	Matches	Example
Universal	every element	*
Compound	one element meeting all the parts	a.btn, .card.featured
Group (comma)	elements matching any of the selectors	h1, h2, h3

Example. For `<a href="/tickets" target="_blank" class="btn primary">Buy</a>`, all of these match: `a`, `.btn`, `.primary`, `a.btn`, `.btn.primary`, `a[target]`, `a[target="_blank"]`. The selector `.btn .primary` does **not** match it: the space asks for a `.primary` element **inside** a `.btn`.

Common mistake. Confusing `.card.featured` (no space: one element with both classes) with `.card .featured` (space: a `.featured` element somewhere inside a `.card`). One space changes the meaning completely, and the wrong one simply matches nothing, with no error.

Combinators: descendant, child and siblings

A combinator joins two selectors by how the elements are related in the HTML tree: anywhere inside (a space), direct child (`>`), the next sibling (`+`) or any later sibling (`~`).

The four combinators

Combinator	Written	Matches the B that is...
Descendant	A B (space)	anywhere inside an A, at any depth
Child	A > B	a direct child of an A (one level down)
Next sibling	A + B	immediately after an A, with the same parent
Later sibling	A ~ B	after an A (not necessarily right after), with the same parent

Example. `.card + .card { margin-top: 1rem; }` puts space **between** consecutive cards but not above the first one: the first card has no `.card` right before it, so it is not matched. `nav > ul` styles the menu list directly inside `<nav>` but not a sub-menu `<ul>` nested deeper.

Common mistake. Writing `.menu > a` for `<nav class="menu"><ul><li><a>...</a></li></ul></nav>`. The link is a great-grandchild of `.menu`, not a direct child, so nothing matches. Use the descendant combinator `.menu a`, or spell out the whole path `.menu > ul > li > a`.

Pseudo-classes: state and position

A pseudo-class such as `:hover`, `:focus`, `:first-child` or `:nth-child(odd)` matches an element by its current **state** or its **position** among its siblings: information that is not written in the HTML.

Pseudo-classes you will use constantly

Pseudo-class	Matches
<code>:hover</code>	while the mouse pointer is over the element
<code>:focus</code>	while the element has keyboard focus (a clicked or tabbed-to input, button or link)
<code>:focus-visible</code>	focus that the browser judges should be visible (typically keyboard navigation, not mouse clicks)
<code>:active</code>	while the element is being pressed
<code>:visited</code>	a link to a page already visited
<code>:first-child</code> / <code>:last-child</code>	the first / last child of its parent
<code>:nth-child(odd)</code> , <code>:nth-child(3n)</code>	children by position: odd ones, every third...
<code>:not(.done)</code>	elements that do not match the selector inside

Example. A sign-in form: `input:focus { border-color: #FF5700; outline: 3px solid #FFD2B8; }` shows clearly which field is being typed in; `button:hover { background: #1A1F6C; }` gives visual feedback; `tr:nth-child(even) { background: #F4F4F8; }` stripes a fixtures table so rows are easy to follow.

Common mistake. Removing the focus ring with `outline: none` and putting nothing in its place: keyboard users can no longer see where they are. If the default ring clashes with your design, **replace** it (`:focus-visible { outline: 3px solid #FF5700; }`). Also watch the space: `button :hover` (with a space) means "a hovered element **inside** a button", not a hovered button.

Cascade and specificity

The cascade: which declaration wins

When several declarations set the **same property** on the **same element**, the cascade chooses one by asking, in this order: **origin and !important, inline style, specificity, and finally source order (the last one wins)**.

The questions the cascade asks, in order (the first one that separates the candidates decides)

#	Question	Winner
1	Origin and importance	your (author) styles beat browser defaults; any !important declaration beats every normal one
2	Inline style?	a style="" attribute beats any selector
3	Specificity	the selector with the higher (a, b, c) score
4	Source order	the declaration that appears last

Example. With `.btn { background: grey; }` followed by `.primary { background: orange; }`, the element `<a class="primary btn">` is **orange**: both selectors score (0,1,0), so the later rule wins. Swap the two rules in the file and it turns grey; swapping the class names in the HTML changes nothing.

Common mistake. Fixing a conflict with !important. It wins today, but the next time you need an override you need another !important, and soon nothing can be changed without one. Look at the Styles pane in DevTools instead: the losing declaration is struck through, so you can see which rule beat it and fix the selector or the order.

Specificity: the (a, b, c) score

Specificity is a three-number score (**a, b, c**) computed for every selector: **a = ids**, **b = classes**, **attribute selectors and pseudo-classes**, **c = type selectors and pseudo-elements**. Scores are compared column by column from the left; the higher one wins.

Worked scores

Selector	(a, b, c)	Why
*	(0,0,0)	universal counts nothing
p	(0,0,1)	one type
nav ul li a	(0,0,4)	four types; the spaces count nothing
.intro	(0,1,0)	one class
li:first-child	(0,1,1)	a pseudo-class counts like a class
a[target]	(0,1,1)	an attribute selector counts like a class
.card > p	(0,1,1)	the > counts nothing
.nav .link:hover	(0,3,0)	two classes plus one pseudo-class
#hero p	(1,0,1)	one id, one type
#hero .intro	(1,1,0)	beats #hero p: equal a, higher b

Example. Three rules colour the same link: `nav a { color: black; }` scores (0,0,2); `.link { color: navy; }` scores (0,1,0); `#main-nav a { color: orange; }` scores (1,0,1). The link is **orange**, wherever the rules appear in the file. Remove the id rule and it is **navy**: one class beats any number of type selectors.

Common mistake. Treating the score as one decimal number ("#hero p is worth 101 points, so eleven classes, worth 110, beat it"). The columns are compared separately and never carry: one id beats any number of classes. The practical lesson is to keep selectors **short and flat** (mostly single classes) so that any rule can be overridden by another single class placed later.

Inheritance: values that flow down the tree

Some properties, mostly text ones such as `color`, `font-family`, `font-size`, `line-height` and `text-align`, are **inherited**: an element with no value of its own takes its parent's. Box properties such as `margin`, `padding`, `border`, `width` and `background` are **not inherited**.

- **Inherited:** `color`, `font-family`, `font-size`, `font-weight`, `line-height`, `text-align`, `list-style`, `cursor`, `visibility`.
- **Not inherited:** `margin`, `padding`, `border`, `width`, `height`, `background`, `display`, `box-shadow`, `border-radius`.

- **Form controls** (button, input, select, textarea) get their own font from the browser, so they ignore the body font unless you write `button, input, select, textarea { font: inherit; }`.

Example. In `<article class="card"><p>Text with <a href="#">a link</a></p></article>`, the rule `.card { color: darkred; border: 1px solid; }` makes the paragraph text dark red (inherited) but draws **one** border, around the card only (not inherited). The link keeps the browser's blue until you write `.card a { color: inherit; }`.

Common mistake. Expecting `.card { color: red; }` to colour the links inside the card, or the login button to use the page font. Both have browser defaults that target them directly, and a direct rule always beats an inherited value. Target them yourself, or use `inherit`.

Box model and flow

The box model: content, padding, border, margin

Every element is drawn as a rectangle of four layers, inside out: **content**, **padding** (space inside the border, painted with the background), **border**, and **margin** (transparent space outside the border that separates the box from its neighbours).

Example. `.card { padding: 16px; border: 1px solid #ddd; margin-bottom: 12px; background: white; }` gives each card 16px of white breathing room around its text, a thin grey frame, and 12px of empty space before the next card.

Common mistake. Using padding to separate two cards. Padding is **inside** the card: each card grows and its background grows with it, but the cards still touch. Space **between** boxes is margin, or gap when the parent is a flex or grid container.

box-sizing: what width really measures

box-sizing **decides which layers width and height measure**: content-box (the default) sizes only the content, so padding and border are **added on top**; border-box makes padding and border fit **inside** the declared size.

`width: 300px; padding: 20px; border: 5px solid;`

	content-box (default)	border-box
Content width	300px	$300 - 2 \times 20 - 2 \times 5 = 250\text{px}$
Visible width (border to border)	$300 + 2 \times 20 + 2 \times 5 = 350\text{px}$	300px
Margin	added outside	added outside

```
*,
*::before,
*::after {
  box-sizing: border-box;
}
```

Example. Two columns with `width: 50%; padding: 20px;` side by side. With content-box each one is 50% + 40px wide, the pair is wider than the container and the second column wraps below the first. With border-box each is exactly 50% and they fit.

Common mistake. Thinking border-box also includes the margin. It does not: `width: 100%; margin: 0 16px;` still overflows its parent by 32px. A block box already fills the available width on its own, so the fix is to remove the width (or use `max-width`).

Margins: centring with auto, and collapsing

`margin: 0 auto` **centres a block that is narrower than its container (it needs a width or max-width)**; vertical margins **between stacked blocks collapse, so the gap is the larger margin, not the sum**.

Example. A sign-in card: `.login { max-width: 420px; margin: 2rem auto; padding: 2rem; border: 1px solid #ddd; }`. On a wide screen it is 420px wide with equal space on both sides; on a 360px phone it simply takes the full width, because `max-width` is a ceiling, not a fixed size.

Common mistake. Using `margin: 0 auto` on a box without a width (a block already fills its container, so there is no leftover space to share) or on an inline element such as `<a>` or `<span>` (horizontal auto margins do nothing on inline boxes). Give the box a `max-width`, or centre it from the parent with Flexbox.

Normal flow and display: block, inline, inline-block, none

`display` sets how a box takes part in layout: **block** boxes stack vertically and fill the available width, **inline** boxes flow inside lines of text, **inline-block** flows in a line but accepts width and height, and **none** removes the element from the page.

How each value behaves

Value	Starts a new line?	Default width	width / height	Vertical margin
block	yes	fills the container	applied	applied (and may collapse)
inline	no	its content	ignored	ignored (padding paints but does not push lines apart)
inline-block	no	its content	applied	applied
none	—	removed from layout and from screen readers	—	—

Example. A form where each `<label>` sits on its own line above its field: `label { display: block; margin-bottom: 4px; }` and `input { display: block; width: 100%; }`. Nav links that need padding and a clickable area larger than the text: `nav a { display: inline-block; padding: 8px 12px; }`.

Common mistake. Setting width, height or margin-top on an `<a>` or a `<span>` and seeing nothing happen: inline boxes size to their text and ignore those properties. Switch to `inline-block` or `block` (or lay out the parent with Flexbox). Also, `visibility: hidden` is not the same as `display: none`: hidden elements are invisible but **keep their space**.

Units and styling

Units: px, %, em, rem, vw/vh and fr

Lengths are either absolute (px) or relative to something: the containing block (%), a font size (em, rem), the viewport (vw, vh) or, in grid tracks only, the free space (fr).

Assuming the browser default root font size of 16px

Unit	Relative to	Example	Typical use
px	nothing (a CSS pixel)	<code>border: 1px solid</code>	borders, shadows, small fixed details
%	the containing block	<code>max-width: 100%</code>	fluid widths, images
em	the element's own font size	<code>padding: 0.5em 1em</code>	spacing that should scale with that element's text
rem	the root (<code><html></code>) font size	<code>font-size: 1.25rem = 20px</code>	font sizes, consistent spacing
vw / vh	1% of the viewport width / height	<code>height: 100vh</code>	full-screen sections
fr	a share of the free space in a grid container	<code>grid-template-columns: 2fr 1fr</code>	grid tracks only

Example. With the default 16px root: `h1 { font-size: 2rem; }` is 32px. A button with `font-size: 0.875rem; padding: 0.5em 1em;` has 14px text and padding of 7px (top/bottom) by 14px (left/right): the `em` padding is computed from the button's own 14px font size, so a bigger button gets proportionally bigger padding automatically.

Common mistake. Nesting `em` font sizes: with `li { font-size: 0.8em; }`, a list inside a list item is $0.8 \times 0.8 = 0.64$ of the original size, and a third level 0.51. `em` compounds through the tree; `rem` always goes back to the root, which is why `rem` is the safer default for font sizes. Also, `fr` is not a general length: `width: 1fr` is invalid and ignored.

Colour, text and decoration

The everyday styling properties almost every page relies on: **colours** (color, background-color), **fonts** (font-family, font-size, font-weight, line-height), **text** (text-align, text-decoration), **lists** (list-style) and **decoration** (border-radius, box-shadow, cursor).

Properties and typical values

Property	What it does	Example
color	text colour (inherited)	color: #222;
background-color	fills content and padding	background-color: #F4F4F8;
font-family	font stack (inherited)	font-family: Arial, sans-serif;
font-size / font-weight	size / boldness	font-size: 1.125rem; font-weight: 700;
line-height	space between lines; unitless = × font size	line-height: 1.5;
text-align	aligns inline content inside the box	text-align: center;
text-decoration	underline (or not) on text, e.g. links	text-decoration: none;
list-style	list markers (bullets, numbers)	list-style: none;
border-radius	rounds the corners	border-radius: 8px;
box-shadow	x-offset y-offset blur colour	box-shadow: 0 2px 8px rgb(0 0 0 / 0.15);
cursor	mouse pointer shape over the element	cursor: pointer;

Example. A nav without bullets: `nav ul { list-style: none; margin: 0; padding: 0; }` and `nav a { text-decoration: none; color: white; }`. A card: `.card { background: white; border-radius: 8px; box-shadow: 0 2px 8px rgb(0 0 0 / 0.15); padding: 1rem; }`.

Common mistake. Writing `text-decoration: none` on a list to remove the bullets: that property removes the **underline** of links. The bullets belong to the list, so it is `list-style: none` on the `<ul>`, usually together with `padding: 0; margin: 0;` to remove the indentation the browser added to make room for them.

Flexbox

Flexbox: container, items and the two axes

`display: flex` makes an element a **flex container** and its direct children **flex items**, laid out along a **main axis** chosen with `flex-direction`; the **cross axis** is perpendicular to it.

What each flex-direction does to the axes

flex-direction	Main axis	Cross axis	<code>justify-content</code> moves items	<code>align-items</code> moves items
row (default)	left → right	top → bottom	horizontally	vertically
row-reverse	right → left	top → bottom	horizontally	vertically
column	top → bottom	left → right	vertically	horizontally
column-reverse	bottom → top	left → right	vertically	horizontally

Example. A site header: `header { display: flex; justify-content: space-between; align-items: center; }` puts the `<h1>` on the left and the `<nav>` on the right (main axis), both vertically centred (cross axis). Inside it, `nav ul { display: flex; gap: 1rem; list-style: none; }` turns the links into a row.

Common mistake. Believing `justify-content` is always horizontal. It works along the **main axis**: after `flex-direction: column` it moves items vertically and `align-items` moves them horizontally. And vertical centring in a column only shows if the container is taller than its content (give it a height or min-height).

Flexbox alignment: justify-content, align-items, gap, wrap

`justify-content` distributes the items along the main axis, `align-items` positions them on the cross axis, `gap` sets the space between them, and `flex-wrap: wrap` lets them flow onto new lines.

justify-content values (main axis)

Value	Leftover space goes...
<code>flex-start</code> (default)	all at the end; items packed at the start

Value	Leftover space goes...
center	half before the first item, half after the last
flex-end	all at the start; items packed at the end
space-between	between items only; first and last touch the edges
space-around	equal space on both sides of every item, so edges get half a gap
space-evenly	equal gaps everywhere, edges included

Example. A tag cloud of `#javascript, #css, #webdev, #accessibility, #performance`: `.tags { display: flex; flex-wrap: wrap; gap: 0.5rem; }` keeps each tag at its natural width and moves to a new line whenever the next tag no longer fits, with the same 0.5rem gap horizontally and vertically.

Common mistake. Spacing items with `margin-right` on each one: the last item gets a useless margin, and wrapped lines get no vertical space. `gap` on the container puts space **only between** items, in both directions. Another one: adding `align-items: center` to a row of cards "to tidy it up" and losing their equal heights, which came from the default stretch.

Flexible sizes: flex-grow, flex-shrink, flex-basis

Each flex item has a **starting size** (`flex-basis`), a **share of the leftover space it may take** (`flex-grow`) and a **share of any overflow it gives back** (`flex-shrink`). The shorthand `flex: grow shrink basis` **sets all three**.

Common flex shorthands

Shorthand	Expands to	Meaning
(default)	0 1 auto	content size, may shrink, never grows
<code>flex: 1</code>	1 1 0%	equal shares of all the space
<code>flex: 2</code>	2 1 0%	twice the share of a <code>flex: 1</code> sibling
<code>flex: auto</code>	1 1 auto	content size plus a share of the leftover
<code>flex: none</code>	0 0 auto	exactly its content size; never grows or shrinks

Example. A search bar: `.search { display: flex; gap: 8px; }, .search input { flex: 1; }, .search button { flex: none; }`. The button keeps its natural width and the input takes **all** the rest, on any screen width. In a 600px container with no gap, two items with `flex: 1` and `flex: 2` become 200px and 400px.

Common mistake. Expecting `flex: 1` items to be equal when one contains a long unbreakable word, a URL or a wide image. A flex item will not shrink below its content's minimum width (`min-width: auto` by default), so it pushes its siblings narrower or overflows. Add `min-width: 0` (and, for text, `overflow-wrap: anywhere`) to the item.

Grid

Grid: tracks, fr and auto-placement

`display: grid` turns an element's children into **grid items** placed in cells formed by column and row **tracks**, sized with `grid-template-columns` / `grid-template-rows` in **px, %, auto or fr (a share of the free space)**.

- `repeat(3, 1fr)` is short for `1fr 1fr 1fr`.
- `minmax(200px, 1fr)`: a track never narrower than 200px, growing up to 1fr.
- `grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));` fits as many 220px-or-wider columns as the container allows, so a card gallery goes from 4 columns to 1 **without any media query** (beyond the slides, but very handy).
- `gap` (or `row-gap` / `column-gap`) sets the space between tracks, never at the outer edges.

Example. The DevNews `<main>` with `display: grid; grid-template-columns: 2fr 1fr; gap: 2rem;` on a 930px container: $930 - 32 = 898\text{px}$ free, so the article column is 598.7px and the sidebar 299.3px, and both keep that 2:1 ratio at any width.

Common mistake. Putting `grid-template-columns: 2fr 1fr` on a `<main>` that has **three** children and expecting "articles left, sidebar right". Auto-placement fills the cells in order: the first child takes the left cell, the second the right cell, and the third wraps to the left cell of row 2. Make the extra child span both columns or place the items explicitly (next card).

Placing grid items: lines, spans and alignment

Grid lines are numbered from 1 at the start edge (and from -1 at the end edge). `grid-column / grid-row` place an item between two lines (1 / 3) or make it span several tracks; `justify-items / align-items` position the content inside each cell.

Placement in a 3-column grid

Declaration	Covers
<code>grid-column: 1 / 2</code>	column 1 only (line 1 to line 2)
<code>grid-column: 1 / 3</code>	columns 1 and 2
<code>grid-column: 1 / -1</code>	all three columns (first line to last line)
<code>grid-column: span 2</code>	two columns, starting wherever auto-placement puts it
<code>grid-column: 2 / span 2</code>	columns 2 and 3
<code>grid-row: span 2</code>	two rows tall

Example. A gallery of six cards in `repeat(3, 1fr)`: give the first card `grid-column: span 2`; `grid-row: span 2`; and it becomes a featured tile covering a 2x2 block, while the remaining cards auto-place around it with no extra HTML.

Common mistake. Counting columns instead of lines: `grid-column: 1 / 2` covers just **one** column. To cover columns 1 and 2 write `1 / 3` (or `1 / span 2`). Another one: using `1 / -1` with only implicit columns: -1 refers to the last line of the **explicit** grid (the one defined by `grid-template-columns`).

Named areas: grid-template-areas

`grid-template-areas` draws the layout as text, one quoted string per row and one name per cell; each item is then placed with `grid-area: name`.

```
.page {
  display: grid;
  grid-template-columns: 240px 1fr;
  grid-template-areas:
    "header header"
    "sidebar content"
    "footer footer";
  gap: 1.5rem;
}
.site-header { grid-area: header; }
.sidebar      { grid-area: sidebar; }
.content      { grid-area: content; }
.site-footer  { grid-area: footer; }

/* narrow screens: redraw the sketch as one column */
@media (max-width: 768px) {
  .page {
    grid-template-columns: 1fr;
    grid-template-areas:
      "header"
      "content"
      "sidebar"
      "footer";
  }
}
```

Example. In the code above, the header and the footer span both columns because their names are repeated across the row. On a narrow screen the media query redraws the sketch as one column and moves the sidebar **below** the content, without touching the HTML.

Common mistake. Defining the areas on the container but forgetting grid-area on one item (typically the header or footer). That item is auto-placed into the first free cell, often squeezed into the sidebar column, and the named area stays empty. Every region in the sketch needs a matching grid-area on exactly one element.

Responsive design

Responsive foundations: viewport tag, fluid widths, fluid images

A responsive page adapts to any screen width. It needs the **viewport** `<meta>` tag, widths that are relative instead of fixed (% , fr, max-width), and images capped with `img { max-width: 100%; height: auto; }`.

```
<!-- in the <head> of EVERY page -->
<meta name="viewport" content="width=device-width, initial-scale=1">

/* styles.css */
img {
  max-width: 100%; /* shrink with the container, never stretch */
  height: auto; /* keep the aspect ratio */
}
.container {
  max-width: 1100px; /* a ceiling, not a fixed width */
  margin: 0 auto;
  padding: 0 1rem;
}
```

Example. An `` in a 360px-wide phone column: with `max-width: 100%; height: auto;` it is drawn 360 × 180, keeping its 2:1 shape. To test, open DevTools and toggle the device toolbar (Ctrl+Shift+M / Cmd+Shift+M) to emulate a phone; without the viewport tag, the emulated phone shows the zoomed-out desktop layout.

Common mistake. Writing `max-width: 100%` but forgetting `height: auto` on an image that has width and height attributes: the width shrinks but the height stays at the attribute value, so the picture is squashed. And using `width: 100%` instead of `max-width: 100%`: small images such as logos and icons are stretched up and blurred.

Media queries and breakpoints

A media query such as `@media (max-width: 768px) { ... }` wraps rules that apply only while a condition about the device is true, most often the viewport width. The width at which the layout changes is called a **breakpoint**.

```
/* Desktop-first: wide layout is the base */
main { display: grid; grid-template-columns: 2fr 1fr; gap: 2rem; }

@media (max-width: 768px) { /* 768px or narrower */
  main { grid-template-columns: 1fr; }
}

/* Mobile-first: one column is the base */
.cards { display: grid; gap: 1rem; }

@media (min-width: 600px) { /* 600px or wider */
  .cards { grid-template-columns: repeat(2, 1fr); }
}
@media (min-width: 1000px) {
  .cards { grid-template-columns: repeat(3, 1fr); }
}
```

Example. A typical news-style page: main is a two-column grid (2fr 1fr); at 768px or less, `@media (max-width: 768px) { main { grid-template-columns: 1fr; } }` turns it into one column, so the `<aside>` drops below the articles. Resize the window (or use the DevTools device toolbar) to check that the switch really happens.

Common mistake. Putting the `@media` block at the **top** of the stylesheet. The base rule `main { grid-template-columns: 2fr 1fr; }` further down has the same specificity and comes later, so it wins at every width and the breakpoint seems broken. Media queries go after the rules they modify. The other classic cause is a missing viewport `<meta>` tag (previous card).

Flexbox or Grid?

Use **Flexbox** when items flow along one line and their content should decide their size; use **Grid** when rows and columns must line up and the container should decide the sizes. Most pages use both: Grid for the page skeleton, Flexbox inside the regions.

Typical choices

Situation	Tool	Why
Logo on the left, nav on the right	Flexbox	one row; <code>justify-content: space-between</code>
Links in a nav bar, buttons in a toolbar	Flexbox	one row of content-sized items with gap
Tags or chips of different lengths	Flexbox + <code>flex-wrap</code>	each item keeps its natural width
Input stretching next to a button	Flexbox	<code>flex: 1</code> on the input
Page skeleton: header / sidebar / main / footer	Grid	two dimensions, named areas
Card gallery whose columns line up	Grid	columns shared by every row
Content + sidebar with a fixed ratio	Grid	<code>grid-template-columns: 2fr 1fr</code>

Example. A club homepage: body (or a wrapper) is a grid with areas header / main / footer; the header is a flex row (crest left, nav right); the fixtures list on the main area is a grid of match cards; each match card is a flex row (home team, score, away team) with `justify-content: space-between`.

Common mistake. Believing one replaced the other ("Grid is the new Flexbox" or "just use Flexbox for everything"). Building a card gallery with wrapped Flexbox gives a last row whose cards stretch wider than the rest; building a nav bar with Grid forces you to count columns every time a link is added. Choose by the shape of the problem.